Multilayer Perceptrons from the Ground Up

Weights, activations, layers, data encoding and the training loop, with runnable code

---

A 20-slide tutorial • companion code: `data.py`, `mlp_numpy.py`, `mlp.py`, `train.py`, `experiments.py`

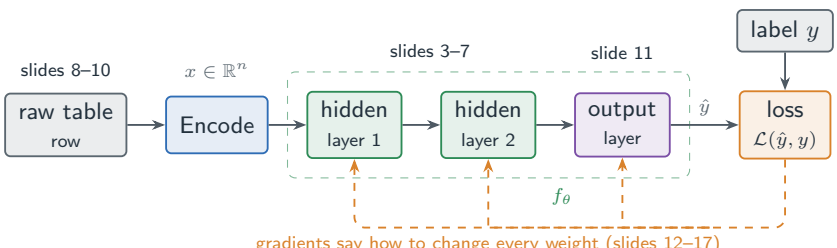
The big picture: a trainable function from inputs to outputs

Supervised learning: we have examples (x, y), an input and the known correct answer, and want a function f_θ with $f_\theta(x) \approx y$ that also works on *new* inputs.

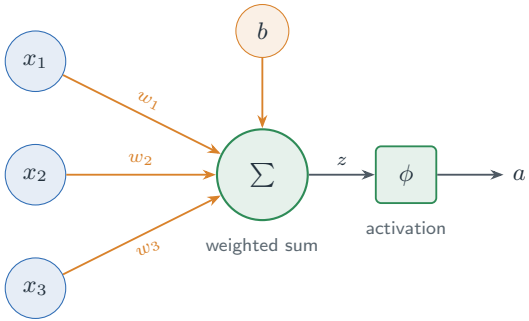
A **multilayer perceptron** (MLP) is the simplest neural network for this job: layers of simple units ("neurons"), each layer feeding the next. θ is the collection of all its **weights**; *training* means adjusting θ until the outputs match the answers.

Two real problems used throughout

- **Classification**: Titanic passenger list, 891 rows. Did this passenger survive? (yes / no)
- **Regression**: Auto MPG, 398 cars. How many miles per gallon does this car get? (a number)



The neuron: weights, bias, activation



$$z = \sum_i w_i x_i + b = w \cdot x + b, \qquad a = \phi(z)$$

By hand

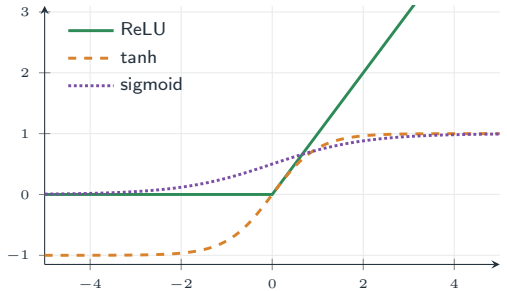
$x = (2, 1)$, $w = (0.5, -1)$, $b = 0.5$:

$z = 0.5 \cdot 2 + (-1) \cdot 1 + 0.5 = \mathbf{0.5}$

$\mathrm{ReLU}(z) = 0.5$, $\mathrm{sigmoid}(z) = 0.622$,
$\tanh(z) = 0.462$

- A **weight** w_i says how much input i matters and in which direction: positive pushes the output up, negative pushes it down, zero ignores the input.
- The **bias** b shifts the threshold: the neuron's output when all inputs are zero.
- The **activation function** ϕ bends the result (next slide).
- **Geometry**: $w \cdot x + b = 0$ is a line (in general a hyperplane). The neuron asks "which side of this boundary is x on, and how far?"
- **Weights are the only thing that learns.** They start as small random numbers; training changes them. "A model with 1,634 parameters" means 1,634 such numbers.
- One neuron alone is the *perceptron* of the 1950s [1]. It can only separate classes with a single straight boundary, which is why we need layers.

Activation functions: where the nonlinearity comes from



	$\phi(z)$	range	$\phi'(z)$
ReLU	$\max(0, z)$	$[0, \infty)$	1 if $z > 0$ else 0
tanh	$\tanh z$	$(-1, 1)$	$1 - \tanh^2 z \ \leq 1$
sigmoid	$1/(1 + e^{-z})$	$(0, 1)$	$\sigma(1 - \sigma) \ \leq \frac{1}{4}$

Why not skip it? Without ϕ, two layers collapse into one:

$$(xW_1)W_2 = x\,(W_1W_2) = xW.$$

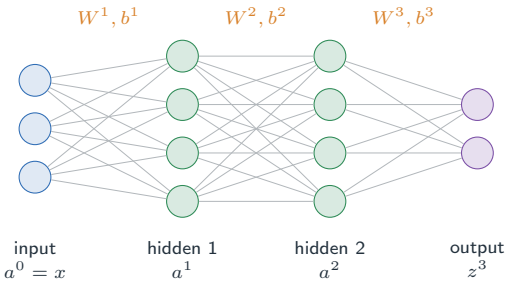
A stack of linear layers is still a linear model, however deep. The nonlinearity is what lets layers build something new on top of each other.

Which one?

- **Hidden layers: ReLU** by default [6]. Cheap, and its slope is exactly 1 for active units, so gradients pass through many layers undiminished.
- **Sigmoid and tanh saturate**: for large $|z|$ the slope is nearly 0 and learning stalls ("vanishing gradients"). Sigmoid's slope is never above $\frac{1}{4}$.
- **Output layer**: chosen by the task, not by taste. None for regression; sigmoid or softmax for classification (slide 11).

A ReLU unit with $z < 0$ outputs 0 and gets no gradient: it is "off" for that input.

Layers: many neurons at once, written as matrices



Forward pass, layer $\ell = 1..L$

$$a^0 = x, \qquad z^\ell = a^{\ell-1} W^\ell + b^\ell, \qquad a^\ell = \phi(z^\ell)$$

The last layer has no ϕ: its raw output z^L goes to the loss.

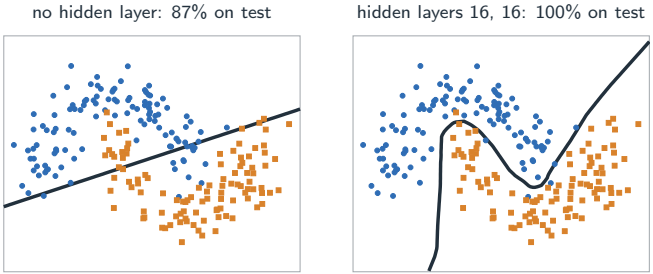
- A **layer** is a set of neurons that all read the same inputs. Stack their weight vectors as the columns of one matrix: $W^\ell \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$.
- **Fully connected** ("dense"): every neuron sees every output of the layer before.
- **Hidden** layers are the ones between input and output. **Depth** = number of layers; **width** = neurons per layer.
- A whole batch at once: rows of $X \in \mathbb{R}^{B \times n_0}$ are examples, and the same formula $XW + b$ handles all B of them in one matrix product.

Counting parameters: layer ℓ has $(n_{\ell-1} + 1)\, n_\ell$. Our Titanic network $15 \to 32 \to 32 \to 2$:

$$16 \cdot 32 + 33 \cdot 32 + 33 \cdot 2 = 512 + 1056 + 66 = \mathbf{1{,}634}.$$

Initialisation. All-zero weights would make every neuron in a layer identical forever. Start random, scaled to the layer: $W^\ell_{ij} \sim \mathcal{N}(0,\, 2/n_{\ell-1})$, $b = 0$ ("He" initialisation, for ReLU [8]; [7] for tanh).

Why hidden layers? A problem no straight line can solve



"Two moons" toy data (220 training points shown). Black curve: where the trained model switches class. Both trained by `experiments.py`; 90 held-out test points.

The left model has 6 parameters, the right one 354. The data and the training procedure are identical; only the hidden layers differ.

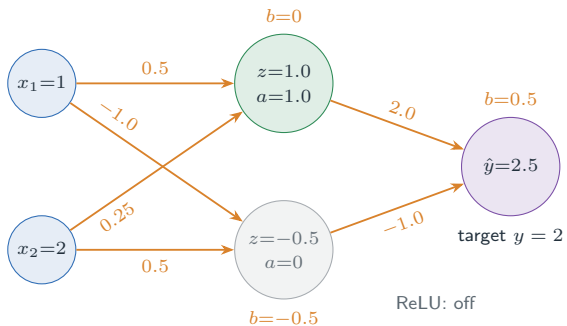
- Each first-layer ReLU neuron contributes **one straight fold** in the input plane.
- The next layer **combines folds** into bent regions; more layers combine those.
- So hidden layers *learn features*: new coordinates in which the problem becomes easy for the final layer.

Universal approximation [4,5]

One hidden layer with enough neurons can approximate any continuous function on a bounded region as closely as you like. It says a good set of weights *exists*; it does not say how many neurons you need or that training will find them.

The classic case is XOR, which no single neuron can compute [2]; `mlp_numpy.py` learns it with 8 hidden units.

A forward pass by hand



A $2 \to 2 \to 1$ regression network. Orange numbers are weights and biases.

Layer 1 ($z^1 = xW^1 + b^1$, then ReLU)

$$z_1^1 = 1(0.5) + 2(0.25) + 0 = \mathbf{1.0} \qquad \Rightarrow a_1^1 = 1.0$$
$$z_2^1 = 1(-1.0) + 2(0.5) - 0.5 = \mathbf{-0.5} \qquad \Rightarrow a_2^1 = 0$$

Output layer (no activation)

$$\hat{y} = 1.0(2.0) + 0(-1.0) + 0.5 = \mathbf{2.5}$$

Loss (squared error)

$$\mathcal{L} = (\hat{y} - y)^2 = (2.5 - 2)^2 = \mathbf{0.25}$$

What to notice

- Prediction is nothing but multiply, add, clip at zero.
- The second hidden neuron is switched off for this input: it contributes nothing to $\hat{y}$.
- The loss turns "how wrong?" into one number. Slide 13 uses it to correct the weights.

Input data I: numerical columns

A network multiplies inputs by weights, so **the size of a number matters**. Raw columns arrive on wildly different scales:

column	meaning	typical values
`age`	years	0.4 … 80
`fare`	ticket price (£)	0 … 512
`weight`	car weight (lb)	1,600 … 5,100
`sibsp`	siblings/spouses aboard	0 … 8

Standardise every numerical column:

$$x' = \frac{x - \mu}{\sigma}$$

with μ, σ the mean and standard deviation of that column. After this each column has mean 0 and spread 1, so one learning rate suits all weights and training converges far more reliably [9].

Example: the training split has $\mu_{\text{age}} = 29.76$, $\sigma_{\text{age}} = 14.77$, so a 32-year-old becomes $(32 - 29.76)/14.77 = \mathbf{0.151}$.

Skewed columns: take a logarithm first

Fares: minimum 0, median 15.05, maximum 512.33. A few huge values would dominate. Use $\ln(1 + x)$, then standardise:

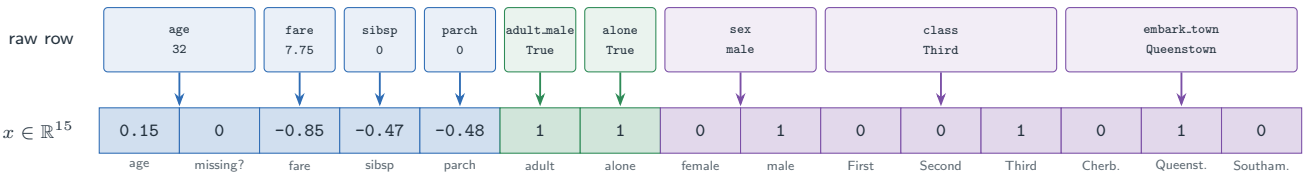
$$7.75 \;\to\; \ln 8.75 = 2.169 \;\to\; \frac{2.169 - 2.986}{0.958} = \mathbf{-0.853}$$

Rule: statistics come from the training split only

Compute μ and σ on training rows, then reuse those same numbers for validation, test and future data. Otherwise information about the test set leaks into training (`TabularEncoder.fit` / `.transform` in `data.py`).

- Counts and integers (`sibsp`, `cylinders`) are numerical too.
- Identifiers (name, ticket number) are *not* features: drop them.
- Alternative scaling: min–max to $[0, 1]$. Fine for bounded data, fragile with outliers.

Input data II: logical, categorical and missing values



Logical (true/false)

Map False $\to 0$, True $\to 1$. Already on a good scale; no standardising needed.

Missing values

134 of 623 training ages are blank. Fill with the mean (which is 0 after standardising) and add a 0/1 column "was missing", so the network can tell a real average from a gap.

Categorical (one of K names)

One-hot: K columns, a single 1 marking the category. `class=Third` $\to (0, 0, 1)$.

A category never seen in training becomes all zeros.

Do not number unordered categories

Coding Cherbourg $= 0$, Queenstown $= 1$, Southampton $= 2$ tells the network that Queenstown lies "between" the others and that Southampton is "twice" Queenstown. Neither is true.

Truly ordered categories (small/medium/large) may use integers. Columns with thousands of categories use learned *embeddings* instead of one-hot.

Labeled data: answers, encodings and the three splits

A **label** is the known correct answer attached to an example. Labels come from records, measurements or human annotators, and the model can never be more reliable than they are.

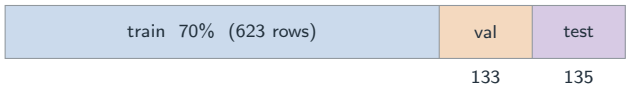
Encoding the label

- **Classification**: class names $\rightarrow$ integer ids. `no` $\rightarrow 0$, `yes` $\rightarrow 1$. With K classes the network gets K outputs, one score per class. The ids are only names for the outputs; the loss treats them as one-hot vectors.
- **Regression**: the label is a number (mpg). Standardise it like an input, train on the standardised value, convert predictions back to real units for reporting.

Always compare with a trivial baseline

62.2% of our Titanic test passengers died, so "always answer `no`" already scores 62.2% accuracy. A model must beat that to have learned anything.

891 labeled Titanic rows, shuffled once



train	the only rows that change the weights
validation	watched during training: when to stop, which learning rate or network size to keep
test	used once, at the end, for the number you report

Leakage: any path by which the answer sneaks into the input.

- The Titanic file has a column `survived` that duplicates the label `alive`. Feed it in and you get 100% accuracy and a useless model.
- Tuning choices on the test set makes the test score optimistic. That is what the validation set is for.

The output layer and the loss: regression vs. classification

The hidden layers are the same for both tasks. Only the last layer and the loss change:

	Regression (predict a number)	**Classification** (predict one of K classes)
output neurons	1	K (scores called *logits*, $z_1..z_K$)
output activation	none: $\hat{y} = z$	softmax: $p_k = e^{z_k} / \sum_j e^{z_j}$, positive, sums to 1
loss (mean over batch)	squared error $(\hat{y} - y)^2$ (MSE)	cross-entropy $-\ln p_y$ (probability given to the true class)
prediction	$\hat{y}$	$\arg\max_k p_k$
report	RMSE, MAE, R^2	accuracy, confusion matrix, precision/recall
gradient at output	$\partial\mathcal{L}/\partial z = 2(\hat{y} - y)$	$\partial\mathcal{L}/\partial z = p - \text{onehot}(y)$

Regression by hand

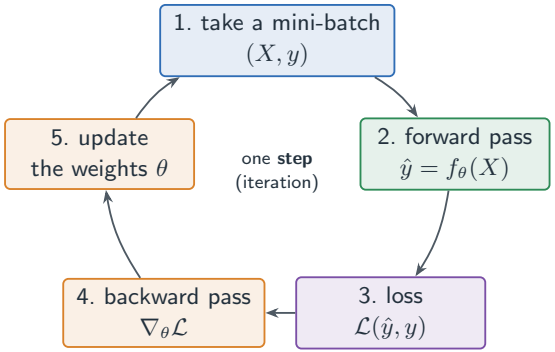
True mpg $(18, 24, 31, 15)$, predicted $(20, 23, 28, 16)$, errors $(2, -1, -3, 1)$:
MSE $= \frac{4+1+9+1}{4} = \mathbf{3.75}$, RMSE $= \sqrt{3.75} = 1.94$ mpg, MAE $= 1.75$ mpg,
$R^2 = 1 - \frac{15}{150} = 0.90$ (share of the variance explained).

Classification by hand

Logits $(2.0,\ 0.5,\ -1.0)$ $\xrightarrow{\text{softmax}}$ $p = (0.786,\ 0.175,\ 0.039)$.
True class is the 2nd: loss $= -\ln 0.175 = \mathbf{1.74}$. Gradient $(0.786,\ -0.825,\ 0.039)$: raise the true score, lower the others.

Two classes: one sigmoid output, $p = 1/(1 + e^{-z})$, is the same model as a 2-way softmax. **Why $-\ln p$?** Confident and wrong is punished heavily.

The training loop



$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Gradient descent: the gradient points uphill on the loss surface; step the other way. η is the learning rate.

In words

1. Initialise the weights randomly.
2. **For each epoch** (one full pass over the training set):
   - shuffle the training rows and cut them into mini-batches;
   - **for each mini-batch**: forward, loss, backward, update;
   - measure the loss on the *validation* set (no updates).
3. Stop when the validation loss has not improved for a while; keep the best weights.
4. Evaluate once on the test set.

- **Step / iteration**: one weight update. **Epoch**: enough steps to see every training row once. 623 rows in batches of 32 = 20 steps per epoch.
- The four knobs of the loop get a slide each: the gradient (13), the learning rate (14), the update rule (15), the batch (16).
- Nothing in the loop depends on the task: swap the loss and the same code does classification or regression (`fit` in `train.py`).

Backpropagation: which way should each weight move?

We need $\partial \mathcal{L}/\partial w$ for *every* weight. The network is a chain of simple steps, so the **chain rule** gives all of them in one backward sweep [3]:

Backward pass (δ^ℓ = error signal at layer ℓ)

$$\delta^L = \partial \mathcal{L}/\partial z^L \qquad \text{from the loss (slide 11)}$$

$$\delta^\ell = \left(\delta^{\ell+1} W^{\ell+1\top}\right) \odot \phi'(z^\ell) \qquad \text{send it back one layer}$$

$$\frac{\partial \mathcal{L}}{\partial W^\ell} = a^{\ell-1\top} \delta^\ell, \qquad \frac{\partial \mathcal{L}}{\partial b^\ell} = \delta^\ell \qquad \text{gradients}$$

- A weight's gradient = (activation feeding it) $\times$ (error at its output).
- Cost: about the same as one forward pass.
- PyTorch does this automatically (`loss.backward()`). `mlp_numpy.py` does it by hand in about 15 lines and matches finite differences to 10^{-10}.

The network from slide 7 ($\hat{y} = 2.5$, $y = 2$, $\mathcal{L} = 0.25$)

$$\delta^2 = 2(\hat{y} - y) = \mathbf{1.0}$$

$$\partial \mathcal{L}/\partial W^2 = a^{1\top} \delta^2 = (1.0,\ 0)$$

$$\delta^1 = (1.0)\,(2.0,\ -1.0) \odot (1,\ 0) = (\mathbf{2.0},\ 0)$$

$$\partial \mathcal{L}/\partial W^1 = x^\top \delta^1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 4 & 0 \end{bmatrix}$$

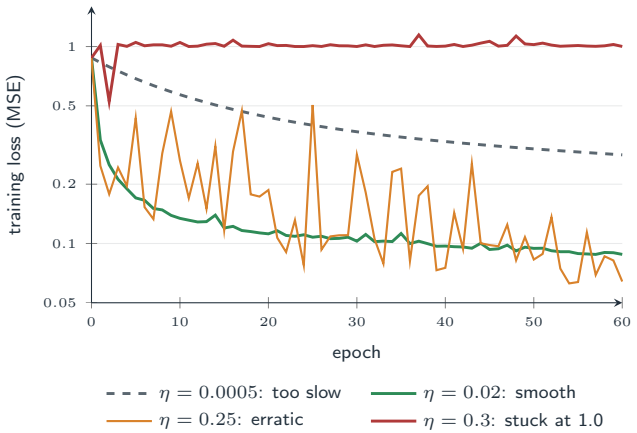
The switched-off neuron has $\phi' = 0$: no gradient reaches its weights this time.

One update, two learning rates

$\eta = 0.01$:	$\hat{y}$: $2.5 \to 2.24$	loss $0.25 \to \mathbf{0.058}$	better
$\eta = 0.1$:	$\hat{y}$: $2.5 \to 0.40$	loss $0.25 \to \mathbf{2.56}$	much worse

The gradient gives the *direction*. It is only valid for a small step, so the step size matters as much as the direction.

The learning rate: the most important knob



Measured: Auto MPG, $10 \to 32 \to 32 \to 1$ network, plain SGD, batch 32, same initial weights. One run per setting.

The learning rate η is the step length in $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$.

- **Too small** (0.0005): every step helps, but after 60 epochs the loss is still 0.282.
- **About right** (0.02): fast, steady descent to 0.088.
- **Large** (0.25): the loss jumps up and down. This run happened to end low (0.064), but it is one small step from failure.
- **Too large** (0.3): an early step overshoots so far that the ReLU units switch off. The network then predicts the mean forever (loss 1.0 for a standardised target). At $\eta = 0.35$ the weights overflow to NaN in the first epoch.

Practice

Search on a log scale $(0.001, 0.003, 0.01, \dots)$ and judge by *validation* loss. If the loss explodes or stalls at a round number, lower η first. Many recipes also shrink η during training (a "schedule").

Optimizers: smarter update rules

All use the same gradient $g = \nabla_\theta \mathcal{L}$ from a mini-batch; they differ in how it becomes a step.

SGD (stochastic gradient descent) [10]

$$\theta \leftarrow \theta - \eta\, g$$

Momentum [11]: keep a running velocity; steady directions build speed, zig-zags cancel.

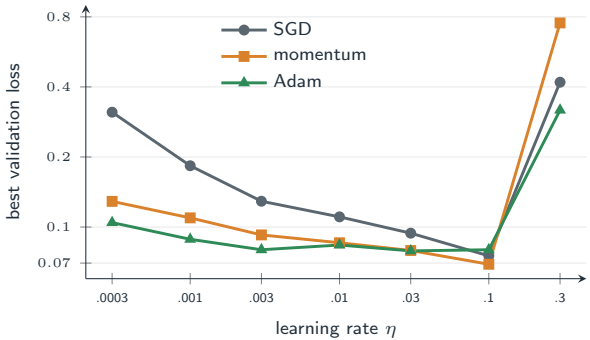
$$v \leftarrow \mu v + g, \qquad \theta \leftarrow \theta - \eta\, v \qquad (\mu = 0.9)$$

Adam [12]: momentum, plus a separate step size per weight from the running mean of g^2.

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2$$

$$\theta \leftarrow \theta - \eta\, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} \qquad (\hat{m}, \hat{v}: \text{ bias-corrected})$$

Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$. Each weight moves about η per step whatever the size of its gradient. **AdamW** [13] adds weight decay done correctly (slide 17).



Measured: same network and data as slide 14, 60 epochs, one run per point.

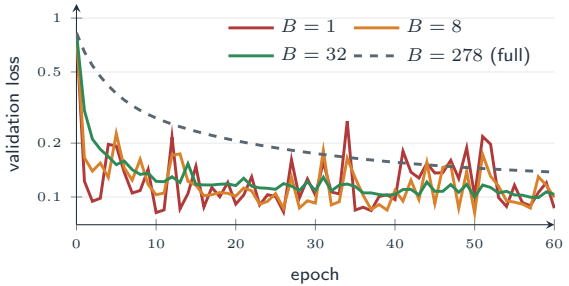
- **Tuned, they tie**: best loss 0.075 (SGD), 0.069 (momentum), 0.079 (Adam) on this small problem.
- **Adam forgives a bad guess**: from $\eta = 0.0003$ to 0.1 its loss stays within 0.079–0.104; SGD ranges from 0.075 to 0.312.
- Hence the usual advice: start with Adam at 10^{-3}; all three degrade sharply at $\eta = 0.3$.

Batching: how many examples per update?

The true gradient averages over *all* training rows. A **mini-batch** of B rows gives a cheap, noisy estimate of it; the noise shrinks like $1/\sqrt{B}$.

$B = N$	**full batch**: exact gradient, one update per epoch
$1 < B < N$	**mini-batch**: the standard choice, typically 16–256
$B = 1$	**stochastic**: one row per update; very noisy, no parallelism

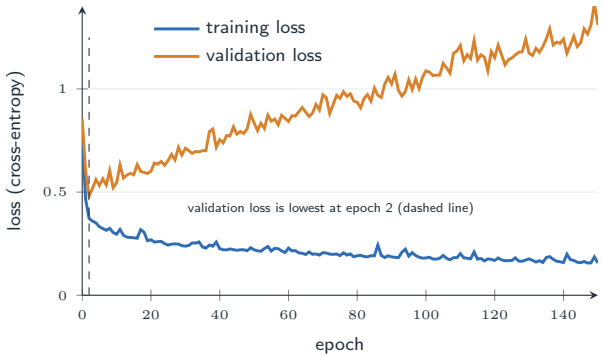
- **Why batches are fast**: a batch is a matrix $X \in \mathbb{R}^{B \times n}$, and $XW + b$ processes all B rows in one vectorised operation.
- **Shuffle every epoch** so batches differ and no ordering in the file (sorted by date, by class…) biases the updates.
- **Noise is not all bad**: it helps the weights escape poor regions, and small batches give many more updates per epoch.
- Batch size and learning rate interact: a larger batch gives a steadier gradient and usually tolerates (and needs) a larger η.



batch B	updates/epoch	ms/epoch	val. loss, 60 epochs
1	278	119	0.087
8	35	19	0.100
32	9	6	0.103
278	1	1	0.138

Measured: Auto MPG (278 training rows), SGD, $\eta = 0.02$, one run each, 2 CPU cores. Per epoch, small batches learn more but cost far more time.

Overfitting: when the training loss lies



Measured: Titanic, deliberately oversized $15 \to 256 \to 256 \to 2$ network (70,402 weights for 623 training rows), Adam, no regularisation.

Training loss keeps falling to 0.156; validation loss bottoms out at 0.482 and climbs to 1.310. The network is **memorising** training rows, noise included, instead of learning the pattern.

Diagnosis

Always plot both curves. A widening gap between training and validation loss is overfitting. Both high and flat is *under*fitting: the model is too small, or η is wrong.

Remedies, roughly in order of cost:

- **Early stopping** [15]: keep the weights from the best validation epoch (`--patience`). Here test accuracy is 85.2% with it, 83.0% when trained to the end.
- **Weight decay**: shrink every weight slightly each step, $\theta \leftarrow \theta - \eta\lambda\theta$, so the fit stays smooth (`--weight_decay`).
- **Dropout** [14]: during training, zero a random fraction of hidden activations each step so no neuron can rely on another (`--dropout`).
- **A smaller network**: fewer weights, less room to memorise.
- **More data**: the only remedy that also makes the model better.

Rule of thumb: pick a network big enough to overfit, then regularise it back.

Code: the model and the training loop in PyTorch

`mlp.py`: build the stack of layers from a list such as `hidden=[32, 32]`

```
        layers, width = [], n_in
        for h in hidden:
            layers.append(nn.Linear(width, h))            # z = a W + b
            layers.append(ACTIVATIONS[activation]())      # a = phi(z)
            width = h
        layers.append(nn.Linear(width, n_out))            # output layer: raw scores, no activation
        self.net = nn.Sequential(*layers)
```

`train.py`, inside `fit()`: the loop of slide 12, with validation and early stopping (lines verbatim; logging and dropout omitted)

```
    for epoch in range(1, epochs + 1):
        model.train()
        for xb, yb in batches(d["X_train"], d["y_train"], batch_size, rng):    # one epoch
            out = model(torch.from_numpy(xb))                 # 1. forward pass
            loss = compute_loss(d["task"], out, torch.from_numpy(yb))  # 2. how wrong?
            opt.zero_grad()
            loss.backward()                                   # 3. backpropagation: all gradients
            opt.step()                                        # 4. update every weight
        row = dict(epoch=epoch, train_loss=split_loss(model, d, "train"),
                   val_loss=split_loss(model, d, "val"), seconds=time.time() - t0)
        history.append(row)
        if row["val_loss"] < best_val - 1e-5:
            best_val, bad_epochs = row["val_loss"], 0
            best_state = copy.deepcopy(model.state_dict())
        else:
            bad_epochs += 1
            if patience and bad_epochs >= patience:           # early stopping
                break
```

The harness on real data: train, validate, test

Classification: `python train.py --task titanic`

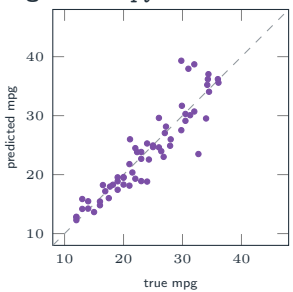
	pred. no	pred. yes
true no	77	7
true yes	12	39

135 test passengers, split 0:
accuracy **85.9%**
majority baseline 62.2%
$15 \to 32 \to 32 \to 2$, 1634 weights,
stopped after 46 epochs

test accuracy, 10 random splits	mean	std
MLP (32, 32)	81.6%	2.9
no hidden layer (logistic regression)	82.7%	2.4

Honest reading: both models clearly beat the baseline, but the hidden layers add nothing here. The MLP wins on only 3 of 10 splits. A single split (85.9%) can flatter either model: with 135 test rows, one standard error is about 3 points.

Regression: `python train.py --task mpg`



61 test cars, split 0:
RMSE **2.84 mpg**
MAE 1.95 mpg, $R^2 = 0.83$
predict-the-mean baseline:
RMSE 6.87

test RMSE (mpg), 10 random splits	mean	std
MLP (32, 32)	2.95	0.75
no hidden layer (linear regression)	3.29	0.41

Here the hidden layers help: fuel economy bends with weight and power, and the MLP is better on 7 of 10 splits. The lesson: always run the linear baseline (`--hidden` with no sizes). On small tables, tree ensembles are often stronger still [17].

References and how to run the code

[1] Rosenblatt. *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.* Psychological Review 65(6), 1958.

[2] Minsky, Papert. *Perceptrons.* MIT Press, 1969.

[3] Rumelhart, Hinton, Williams. *Learning Representations by Back-propagating Errors.* Nature 323, 1986.

[4] Cybenko. *Approximation by Superpositions of a Sigmoidal Function.* Mathematics of Control, Signals and Systems 2, 1989.

[5] Hornik, Stinchcombe, White. *Multilayer Feedforward Networks are Universal Approximators.* Neural Networks 2, 1989.

[6] Nair, Hinton. *Rectified Linear Units Improve Restricted Boltzmann Machines.* ICML 2010.

[7] Glorot, Bengio. *Understanding the Difficulty of Training Deep Feedforward Neural Networks.* AISTATS 2010.

[8] He, Zhang, Ren, Sun. *Delving Deep into Rectifiers.* ICCV 2015. arXiv:1502.01852

[9] LeCun, Bottou, Orr, Müller. *Efficient BackProp.* In Neural Networks: Tricks of the Trade, Springer, 1998.

[10] Robbins, Monro. *A Stochastic Approximation Method.* Annals of Mathematical Statistics 22, 1951.

[11] Polyak. *Some Methods of Speeding up the Convergence of Iteration Methods.* USSR Computational Mathematics and Mathematical Physics 4(5), 1964.

[12] Kingma, Ba. *Adam: A Method for Stochastic Optimization.* ICLR 2015. arXiv:1412.6980

[13] Loshchilov, Hutter. *Decoupled Weight Decay Regularization.* ICLR 2019. arXiv:1711.05101

[14] Srivastava, Hinton, Krizhevsky, Sutskever, Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting.* JMLR 15, 2014.

[15] Prechelt. *Early Stopping – But When?* In Neural Networks: Tricks of the Trade, Springer, 1998.

[16] Goodfellow, Bengio, Courville. *Deep Learning.* MIT Press, 2016. deeplearningbook.org

[17] Grinsztajn, Oyallon, Varoquaux. *Why Do Tree-based Models Still Outperform Deep Learning on Typical Tabular Data?* NeurIPS 2022. arXiv:2207.08815

[18] Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS 2019. arXiv:1912.01703

[19] Data: Titanic and Auto MPG tables as distributed in *seaborn-data* (github.com/mwaskom/seaborn-data); Auto MPG is from the UCI Machine Learning Repository.

Run it (Python, NumPy, PyTorch; about a minute on a laptop CPU)

`python test_mlp.py`	six checks, incl. backprop vs. finite differences
`python walkthrough.py`	the hand calculations (slides 3, 7–9, 11, 13)
`python mlp_numpy.py`	from-scratch network learns XOR
`python train.py --task titanic`	the harness; `--help` lists every knob
`python experiments.py`	figures for slides 6, 14–17, 19

Exercises. Feed the leaky `survived` column in and watch accuracy hit 100%. Remove standardisation and retrain. Replace ReLU by sigmoid in a 6-layer network.